

Arrays Basics & Fundamental Methods

Mastering data structures: Declaration, indexing, modification, iteration & core array operations ([push](#), [pop](#), [slice](#), [splice](#))

🕒 Duration: 30 Mins 📁 Part 1 of 2 Series

Today's Learning Agenda

1. Core Foundations

Why we need arrays, train analogy, zero-indexed access, and array length property.

2. End Operations

Adding & removing from the end using `push()` and `pop()` with real-world cart examples.

3. Start Operations

Adding & removing from the start using `unshift()` and `shift()` + performance insights.

4. Splitting & Mutating

Deep dive into non-mutating `slice()` vs mutating `splice()` method.

5. Searching Arrays

Finding items and position checks using `indexOf()` and `includes()`.

6. Practice & Recap

Hands-on practice task, key recap summary, and preview of modern array methods.

Why Do We Need Arrays in Real Applications?

✗ The Single-Value Limitation

- ⚠ Standard variables hold only one value at a time
- ⚠ Managing 100 products requires 100 separate variables
- ⚠ Unscalable and messy code structure

✓ The Array Solution

- ✓ Store thousands of values in a single ordered list
- ✓ Essential for To-Do lists, product catalogs & user feeds
- ✓ Used in literally every real-world web application



Real-World Analogy: Think of an E-commerce Shopping Cart — items are added, removed, reordered, and scanned dynamically as an array!

What is an Array? (The Train Analogy)



Array Name:
fruits

"Apple"

Index 0

"Banana"

Index 1

"Mango"

Index 2

Single Train Name

The entire collection is stored under one variable name, eliminating individual variables.

Compartment Numbers

Each item occupies a specific compartment identified by its unique numeric position (index).

Creating Arrays & Accessing Elements

```
// 1. Creating an Array using literal notation [] let fruits = ["Apple", "Banana", "Mango", "Orange"]
```

 **Rule #1 for Beginners:** Array indexing ALWAYS starts at 0, not 1! Element 1 is at index 0.

Undefined Access & Heterogeneous Arrays

```
// Accessing Non-Existent Index let fruits =
```

```
// Arrays Hold Mixed Data Types let mixedArr
```

Key Takeaway

JavaScript arrays are dynamically typed and loose. They can store numbers, strings, booleans, objects, and even nested arrays inside a single array structure.

Array Length & Looping with For Loop

```
let fruits = ["Apple", "Banana", "Mango", "Orange"]; // 1. .length Property (Returns total count of
```

.length is a Property

Do NOT add brackets `()`. It is a property, not a method function call.

→ Index Range

The valid index range for any array is always from `0` to `length - 1`.

Common Mistake: Off-By-One Boundary Error

```
// ❌ WRONG: Using ≤ with .length for (let i = 0; i ≤ fruits.length; i++) {
```

```
// ✅ CORRECT: Use < with .length for (let i = 0; i < fruits.length; i++) {
```

Why does this happen? An array of length 4 has indices 0, 1, 2, 3. When `i = 4`, accessing `fruits[4]` returns undefined!

End Operations: push() and pop()

```
let fruits = ["Apple", "Banana"]; // push(): Adds one or more elements to the END fruits.push("Mang
```

BEFORE pop():

Apple

Banana

Grapes



AFTER pop():

Apple

Banana

Grapes

Shopping Cart State Trace

```
let cart = ["Shirt", "Shoes", "Watch"]; console.log("Initial Cart:", cart); // User adds an item to
```

State 1: Initial

["Shirt", "Shoes", "Watch"]

State 2: push("Belt")

["Shirt", "Shoes", "Watch", "Belt"]

State 3: pop()

["Shirt", "Shoes", "Watch"]

Start Operations: unshift() and shift()

```
// unshift(): Adds element to the START of the array let fruits = ["Banana", "Mango"]; fruits.unshi
```

unshift("Apple"):

Apple

Banana

Mango

shift():

Apple

Banana

Mango

Basic Methods Matrix & Performance Note

Method	Action	Position	Returns
<code>push()</code>	Adds item(s)	END	New array length
<code>pop()</code>	Removes item	END	The removed element
<code>unshift()</code>	Adds item(s)	START	New array length
<code>shift()</code>	Removes item	START	The removed element

⚡ **Interview Insights — Performance Distinction**
 ⚡ `push/pop` are faster than `unshift/shift` because adding/removing at the start forces JavaScript to re-index every single remaining element in memory!

slice(): Extracting Sub-arrays Safely

```
let fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]; // slice(startIndex, endIndex) – END
```

🛡 Non-Mutating (Safe)

`slice()` NEVER changes the original array. It returns a brand new array copy containing the extracted items.

🚫 Excludes End Index

`slice(1, 3)` extracts index 1 and index 2. Index 3 is NOT included in the final result!

slice(): Useful Arguments & Array Cloning

```
let fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]; // 1. Single argument: Extract from
```

💡 **Developer Pattern:** Calling `arr.slice()` with zero arguments is a classic method to clone an array before applying destructive operations.

splice(): Modifying Array Contents In-Place

```
// Syntax: splice(startIndex, deleteCount, item1, item2, ... ) // 1. REMOVE elements let f1 = ["App1
```

indexOf() & includes(): Locating Items

```
// indexOf(): Returns index OR -1 let fruits
```

```
// includes(): Returns boolean let fruits =
```

📍 Use indexOf()

When you need the exact numeric position of an element to modify or splice it later.

❓ Use includes()

When you only need a clean `true/false` check for conditional statements.

slice() vs splice() — Direct Comparison

Feature	slice()	splice()
Mutates Original Array?	❌ No (Original Safe)	✅ Yes (Changes Original)
Return Value	New array (Extracted items)	Array of removed elements
Can Add Elements?	❌ No	✅ Yes
Primary Purpose	Copy or extract portion	Add, remove, or replace in-place

💡 **Memory Trick: "Slice = Safe (Original Intact) | sPlice = Powerful (Original Changed)"**

Live Practice Task (Hands-On)

- 1 Create an array of 5 favorite movies. Log the 2nd and last movie using their index.
- 2 Use `push()` to add a new movie, then `pop()` to remove the last one.
- 3 Use `unshift()` to add a movie at the start, then `shift()` to remove it.
- 4 Use `slice()` to extract the middle 2 movies into a new array, and confirm original is unchanged.
- 5 Use `splice()` to remove one movie and replace it with two new ones.
- ★ **Bonus:** Use `includes()` to check if "Inception" is in your movie list.

Day 6 Key Takeaways Recap

Arrays & Indexing

- ✓ Stores multiple values under 1 name
- ✓ Indexing starts at 0
- ✓ Range is 0 to `length - 1`

Basic Methods

- ✓ `push` / `pop` work on the END
- ✓ `unshift` / `shift` work on START
- ✓ `push`/`pop` are faster

Slicing & Searching

- ✓ `slice()` = Safe & non-mutating
- ✓ `splice()` = Powerful & mutates
- ✓ `includes()` = Boolean check

Homework & Next Session Preview

Homework Assignment

Complete all 6 practice task steps in VS Code.

Key Challenge: Perform the same extraction task using both `slice()` and `splice()`, and observe the mutation difference in console.

Day 7 Preview: Array Methods Part 2

In the next session, we cover the higher-order modern array methods essential for modern React development:

`map()``filter()``reduce()``forEach()`

✓ DAY 6 COMPLETED

Great Job Today!

Share your practice task output in the comments below. Like & subscribe if you found this helpful!

See You In Day 7 — Allah Hafiz! 🙌